

# LAPORAN MILESTONE 1

## Pengembangan Enterprise AI Copilot untuk Otomasi Customer Service E-Commerce Berbasis Agentic AI dan Retrieval-Augmented Generation

|                          |                                                                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mata Kuliah</b>       | 10S3001 - Kecerdasan Buatan (Artificial Intelligence)                                                                               |
| <b>Kode Tugas</b>        | Tugas 1 (Milestone 1 - W02)                                                                                                         |
| <b>Kelompok</b>          | Grup 17                                                                                                                             |
| <b>Anggota</b>           | 1. Adithya Philip Jona Putra Silaban - 12S24029<br>2. Immanuel Alexander Tambunan - 12S24034<br>3. Mutiara Y.H. Sianturi - 12S24045 |
| <b>Repositori GitHub</b> | <a href="https://github.com/ImmanuelTambunan/ecommerce-cs-ai-agent">https://github.com/ImmanuelTambunan/ecommerce-cs-ai-agent</a>   |
| <b>Dosen Pengampu</b>    | Samuel Indra Gunawan Situmeang, S.T.I.,M.Sc                                                                                         |

# 1. Business Problem Framing

## 1.1 Profil Organisasi

BeliCepat.id merupakan platform *marketplace e-commerce* skala menengah-berkembang yang melayani lebih dari 50.000 pelanggan aktif harian dengan perputaran transaksi yang dinamis. Dalam menjaga kepuasan serta retensi pengguna, operasional layanan pelanggan (*customer service*) BeliCepat.id menangani rata-rata 5.000 tiket keluhan dan permintaan informasi per hari. Untuk mengelola aliran tiket tersebut, struktur layanan pelanggan dibagi ke dalam tiga divisi spesialis, yaitu Divisi Order & Pengiriman, Divisi Retur & Refund, serta Divisi Teknis dan Akun. Setiap divisi memiliki Standard Operating Procedure (SOP), basis pengetahuan operasional, serta batasan wewenang penanganan yang berbeda. Pembagian struktur ini dirancang untuk memastikan bahwa setiap kendala diselesaikan oleh personel yang memiliki kompetensi domain yang relevan. Namun, seiring dengan pesatnya volume transaksi dan interaksi antardivisi, ketergantungan pada koordinasi manual memunculkan friksi operasional yang menghambat pencapaian standar layanan yang optimal. BeliCepat.id dalam proyek ini digunakan sebagai studi kasus/skenario e-commerce yang disimulasikan untuk kebutuhan perancangan dan pengembangan prototype sistem.

## 1.2 Identifikasi Pain Points

Ketergantungan pada pemilahan dan pengelolaan tiket secara manual menimbulkan tiga kendala operasional utama pada alur layanan pelanggan BeliCepat.id. Pertama, waktu respons awal (*first response time*) tiket pelanggan berkisar antara 18 hingga 24 jam karena proses triase, pembacaan keluhan, dan penugasan tiket ke divisi terkait masih dilakukan secara manual oleh staf administrasi. Kedua, tingginya angka salah rute penugasan (*misrouting*) tiket yang mencapai 30% dari total tiket harian, di mana ketidaktepatan disposisi antara divisi Order, Refund, dan Teknis memicu fenomena lempar tiket antardivisi (*ticket ping-pong*), memperpanjang siklus penyelesaian masalah, serta menurunkan skor kepuasan pelanggan (*CSAT*). Ketiga, tim operasional mengalami inelastisitas kapasitas penanganan saat terjadi lonjakan transaksi musiman seperti program promosi *flash sale*, di mana volume tiket keluhan dapat melonjak hingga lima kali lipat mencapai 25.000 tiket per hari. Pada kondisi lonjakan ekstrem ini, kapasitas agen manusia tidak memadai untuk menyerap beban kerja secara instan, mengakibatkan penumpukan antrean tiket (*backlog*), pelanggaran batas waktu *Service Level Agreement* (SLA), serta potensi kehilangan pelanggan akibat keterlambatan resolusi.

## 1.3 Justifikasi Solusi Kecerdasan Buatan

Solusi konvensional berupa penambahan agen manusia (*scaling headcount*) membutuhkan biaya operasional linear yang sangat tinggi dan tidak efisien untuk menghadapi fluktuasi beban berkala. Di sisi lain, sistem otomasi berbasis aturan deterministik (*rule-based* atau percabangan logika *if-else*) terbukti tidak memadai karena tidak mampu memahami variasi bahasa alami, kesalahan pengetikan (*typo*), serta keluhan multitujuan (misalnya keluhan keterlambatan pengiriman yang disertai permohonan pengembalian dana). Pendekatan berbasis aturan statis rentan mengalami kegagalan penafsiran konteks, sehingga risiko salah rute tiket tetap tidak terselesaikan. Penerapan *Enterprise AI Copilot* berbasis *Agentic AI* dan *Retrieval-Augmented Generation* (RAG) menjadi solusi komputasional yang tepat dan adaptif. Melalui arsitektur *Agentic AI*, sistem memiliki kapabilitas penalaran kontekstual untuk menguraikan intent pelanggan yang kompleks, mengambil keputusan otonom mengenai divisi tujuan yang tepat secara *real-time*, serta mengeksekusi tindakan berbasis alur kerja. Sementara itu, komponen RAG memungkinkan asisten cerdas ini mengakses basis data pengetahuan internal, dokumen SOP, dan status pesanan BeliCepat.id secara dinamis guna memberikan draf resolusi yang lebih faktual serta membantu mengurangi risiko kesalahan informasi sehingga memangkas waktu tunggu dari hitungan jam menjadi hitungan detik dengan skalabilitas sistem yang elastis.

## 2. Spesifikasi Formal PEAS

Spesifikasi PEAS (Performance measure, Environment, Actuators, Sensors) berikut merumuskan batasan operasional agen cerdas yang akan dikembangkan untuk menangani proses klasifikasi dan perutean tiket layanan pelanggan BeliCepat.id.

| Elemen                     | Deskripsi                                                                                                                                                                                                                                                  |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Performance Measure</b> | Waktu respons rata-rata penanganan tiket, akurasi klasifikasi dan perutean tiket ke divisi yang tepat, tingkat kepuasan pelanggan (Customer Satisfaction Score/CSAT), serta tingkat eskalasi berulang (tiket yang harus dipindahkan lebih dari satu kali). |
| <b>Environment</b>         | Platform layanan pelanggan BeliCepat.id yang mencakup kanal live chat, tiket keluhan, dan email, tempat agen berinteraksi dengan pelanggan serta staf customer service.                                                                                    |
| <b>Actuators</b>           | Mengirimkan balasan otomatis kepada pelanggan, merutekan atau mengeskalasikan tiket ke divisi/level penanganan berikutnya, menandai prioritas tiket, dan menutup tiket yang telah terselesaikan.                                                           |

|                |                                                                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sensors</b> | Teks pesan atau keluhan pelanggan, riwayat transaksi dan status pesanan, kategori produk terkait, serta metadata waktu masuknya tiket. |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|

## 2.1 Klasifikasi Karakteristik Lingkungan (Environment)

Lingkungan operasional agen turut dianalisis berdasarkan karakteristik berikut agar strategi perancangan agen dapat disesuaikan secara tepat.

| Karakteristik | Klasifikasi          | Justifikasi                                                                                                                                    |
|---------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Observability | Partially Observable | Sistem tidak dapat mengetahui secara pasti niat atau emosi sebenarnya dari pelanggan, hanya dapat menafsirkannya dari teks keluhan yang masuk. |
| Determinism   | Stochastic           | Pola kedatangan tiket dan reaksi pelanggan terhadap penanganan bersifat tidak pasti dan sulit diprediksi sepenuhnya.                           |
| Episodisitas  | Sequential           | Keputusan perutean tiket pada satu waktu memengaruhi beban kerja staf dan keputusan perutean tiket berikutnya.                                 |
| Dinamika      | Dynamic              | Volume dan jenis tiket berubah seiring waktu, terutama meningkat tajam pada periode promosi atau flash sale.                                   |
| Representasi  | Discrete             | Status tiket, kategori keluhan, dan level eskalasi merupakan variabel diskret dengan nilai yang terbatas dan terdefinisi.                      |
| Jumlah Agen   | Multi-agent          | Lingkungan melibatkan interaksi banyak staf customer service dan banyak pelanggan secara bersamaan.                                            |

## 2.2 Penyederhanaan Model Search

Karakteristik partially observable, stochastic, dan multi-agent pada bagian sebelumnya menggambarkan lingkungan operasional customer service secara keseluruhan. Untuk kebutuhan baseline search pada Milestone 1, lingkungan tersebut disederhanakan menjadi model routing tiket berbentuk graf dengan kondisi yang diketahui, transisi yang deterministik, dan satu agen pengambil keputusan. Penyederhanaan ini dilakukan agar permasalahan dapat diformulasikan sebagai path-cost search problem dan diselesaikan menggunakan Uniform Cost Search (UCS).

## 3. Formulasi Ruang Keadaan dan Algoritma Search

### 3.1 Formulasi Formal Masalah Pencarian

Permasalahan perutean tiket keluhan pelanggan pada sistem Enterprise AI Copilot BeliCepat.id diformulasikan sebagai search problem dengan lima komponen formal berikut.

#### a. Ruang Status (X)

$$X = \{\text{Diterima, Level\_1, Level\_2, Supervisor, Selesai}\}$$

Setiap elemen  $x \in X$  merepresentasikan tahap penanganan tiket dalam alur eskalasi layanan pelanggan, mulai dari tiket baru masuk ke sistem (*Diterima*) hingga tiket dinyatakan tuntas ditangani (*Selesai*).

#### b. Himpunan Aksi (A)

$$A = \{\text{tangani\_langsung, eskalasi\_ke\_level\_berikut, eskalasi\_prioritas}\}$$

Ketiga aksi ini merepresentasikan keputusan yang dapat diambil oleh *routing agent* pada setiap status: menyelesaikan tiket pada level saat itu (*tangani\_langsung*), meneruskan tiket secara berjenjang ke level penanganan berikutnya (*eskalasi\_ke\_level\_berikut*), atau melakukan eskalasi langsung ke *Supervisor* untuk kasus yang membutuhkan penanganan prioritas (*eskalasi\_prioritas*).

#### c. Fungsi Transisi (T)

$$T: X \times A \rightarrow X$$

Fungsi T memetakan pasangan (status saat ini, aksi yang dipilih) ke status hasil berikutnya. Karena struktur masalah bersifat deterministik dan dapat direpresentasikan sebagai graf berarah, fungsi transisi ini secara ekuivalen dinyatakan sebagai *successor function*  $Succ(x)$ , yaitu himpunan status tetangga yang dapat dicapai langsung dari status  $x$  beserta biaya transisinya masing-masing.

#### d. Status Awal dan Uji Tujuan (Goal Test)

Status awal didefinisikan sebagai  $X_0 = \text{Diterima}$ , yaitu kondisi saat tiket baru pertama kali masuk ke sistem. Uji tujuan dirumuskan sebagai:

$$\text{Goal}(x) = (x = \text{Selesai})$$

Perlu ditegaskan bahwa kriteria solusi yang dicari bukan sekadar tercapainya status *Selesai* (*reachability*), melainkan **jalur menuju Selesai dengan akumulasi biaya minimum** — sehingga masalah ini termasuk kategori *optimal path-cost search problem*, bukan sekadar *any-path search*.

#### e. Fungsi Biaya (C)

$$C: X \times A \times X \rightarrow \mathbb{R}^+$$

Fungsi biaya merepresentasikan estimasi waktu penanganan (dalam satuan menit) pada setiap transisi antar-status. Nilai biaya bersifat non-negatif dan tetap (*static edge weight*). Dengan penyederhanaan model search yang telah dijelaskan pada Bagian 2.2, permasalahan ini dimodelkan sebagai deterministic, fully observable, single-agent path-cost search problem.

### 3.2 Representasi Graf Masalah

Formulasi formal pada bagian 3.1 direpresentasikan sebagai graf berarah berbobot  $G=(X,E,C)$ , dengan  $X$  sebagai simpul (*node*) dan  $E$  sebagai sisi (*edge*) berikut bobot biayanya:

| Simpul Asal | Simpul Tujuan | Bobot (menit) | Aksi Terkait              |
|-------------|---------------|---------------|---------------------------|
| Diterima    | Level_1       | 5             | tangani_langsung          |
| Level_1     | Selesai       | 10            | tangani_langsung          |
| Level_1     | Level_2       | 15            | eskalasi_ke_level_berikut |
| Level_2     | Selesai       | 12            | tangani_langsung          |
| Level_2     | Supervisor    | 20            | eskalasi_ke_level_berikut |
| Supervisor  | Selesai       | 8             | tangani_langsung          |

Struktur graf ini bersifat **asiklik (Directed Acyclic Graph/DAG)** dan seluruh transisi mengarah maju menuju status *Selesai* — tidak terdapat jalur mundur (misalnya dari *Level\_2* kembali ke *Diterima*). Sifat ini menjamin bahwa algoritma pencarian tidak akan terjebak pada siklus tak berhingga, sehingga properti *completeness* dari algoritma UCS pada graf ini terjamin secara struktural.

Terdapat tiga kemungkinan jalur dari status *Diterima* menuju *Selesai*, dengan total biaya sebagai berikut:

| Jalur                                               | Perhitungan Biaya | Total (menit) |
|-----------------------------------------------------|-------------------|---------------|
| Diterima → Level_1 → Selesai                        | 5 + 10            | 15            |
| Diterima → Level_1 → Level_2 → Selesai              | 5 + 15 + 12       | 32            |
| Diterima → Level_1 → Level_2 → Supervisor → Selesai | 5 + 15 + 20 + 8   | 48            |

Dari ketiga alternatif tersebut, jalur pertama merupakan solusi optimal dengan biaya minimum sebesar 15 menit.

### 3.3 Implementasi Algoritma Uniform Cost Search

Uniform Cost Search (UCS) adalah algoritma *uninformed search* yang mengekskansi simpul berdasarkan biaya kumulatif  $g(n)$  terkecil dari status awal, berbeda dengan BFS yang berbasis kedalaman atau DFS yang berbasis urutan penelusuran. Struktur data yang digunakan adalah **priority queue (min-heap)**, di mana simpul dengan biaya kumulatif terkecil selalu berada pada posisi terdepan untuk diekspansi lebih dulu.

#### a. Penggunaan struktur data heapq

Implementasi UCS pada proyek ini menggunakan modul heapq sebagai priority queue berbasis binary min-heap. Setiap elemen yang dimasukkan ke *frontier* berupa tuple (biaya\_kumulatif, status, path). Karena heapq Python secara default mengurutkan tuple berdasarkan elemen pertama, maka simpul dengan biaya kumulatif terendah otomatis menempati puncak heap (heap[0]) dan akan diambil lebih dulu oleh operasi heapq.heappop().

#### b. Pseudocode implementasi

```
import heapq

def ucs(graph, start, goal):
    frontier = [(0, start, [start])] # (biaya, status, path)
    visited = set()

    while frontier:
        cost, node, path = heapq.heappop(frontier)
```

```

if node == goal:
    return path, cost
if node in visited:
    continue
visited.add(node)

for neighbor, weight in graph[node].items():
    if neighbor not in visited:
        heapq.heappush(frontier, (cost + weight, neighbor, path + [neighbor]))

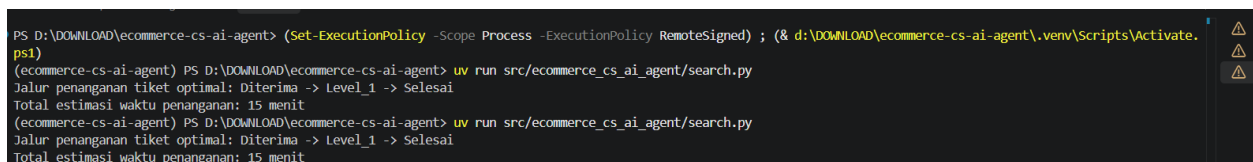
return None, float('inf')

```

### c. Ilustrasi eksekusi (trace) pada studi kasus

| Iterasi | Simpul Di- <i>pop</i> | Biaya Kumulatif | Isi Frontier Setelah Ekspansi             |
|---------|-----------------------|-----------------|-------------------------------------------|
| 1       | Diterima              | 0               | (5, Level_1)                              |
| 2       | Level_1               | 5               | (15, Selesai), (20, Level_2)              |
| 3       | Selesai               | 15              | <i>Goal tercapai — pencarian berhenti</i> |

Karena simpul *Selesai* pertama kali di-*pop* dari heap merupakan simpul dengan biaya kumulatif terkecil di antara seluruh simpul pada *frontier* saat itu (bukan sekadar simpul goal pertama yang ditemukan), maka jalur yang dihasilkan **terjamin optimal**. Properti ini berlaku selama seluruh bobot *edge* pada graf bernilai non-negatif — syarat yang terpenuhi pada kasus BeliCepat.id karena seluruh bobot merepresentasikan estimasi waktu (menit) yang secara alamiah selalu bernilai positif.



```

PS D:\DOWNLOAD\ecommerce-cs-ai-agent> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\DOWNLOAD\ecommerce-cs-ai-agent\venv\Scripts\Activate.ps1)
(ecommerce-cs-ai-agent) PS D:\DOWNLOAD\ecommerce-cs-ai-agent> uv run src/ecommerce_cs_ai_agent/search.py
Jalur penanganan tiket optimal: Diterima -> Level_1 -> Selesai
Total estimasi waktu penanganan: 15 menit
(ecommerce-cs-ai-agent) PS D:\DOWNLOAD\ecommerce-cs-ai-agent> uv run src/ecommerce_cs_ai_agent/search.py
Jalur penanganan tiket optimal: Diterima -> Level_1 -> Selesai
Total estimasi waktu penanganan: 15 menit

```

**Gambar 1.** Hasil eksekusi baseline Uniform Cost Search pada studi kasus BeliCepat.id

#### d. Catatan mengenai heuristik

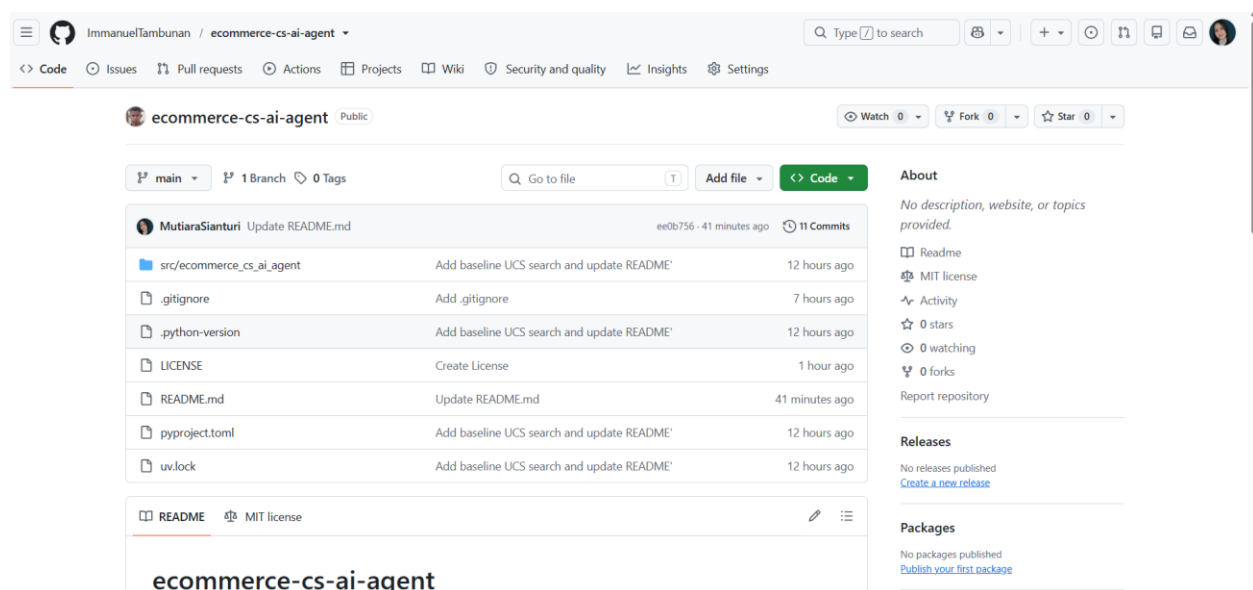
UCS dapat dipandang sebagai kasus khusus algoritma A\* dengan  $h(n)=0$  untuk seluruh simpul, sehingga fungsi evaluasi  $f(n)=g(n)+h(n)$  tereduksi menjadi  $f(n)=g(n)$ . Heuristik nol ini secara trivial memenuhi syarat:

- **Admissible:**  $h(n) \leq h^*(n)$ , karena nilai 0 tidak pernah melebihi biaya asli sesungguhnya.
- **Consistent (monotonic):**  $h(n) \leq c(n, n') + h(n')$ , karena  $0 \leq c(n, n') + 0$  selalu benar untuk bobot  $c \geq 0$ .

Dengan demikian, meskipun implementasi yang digunakan adalah UCS murni (tanpa heuristik informatif), algoritma ini tetap memenuhi kerangka teoritis A\* dengan heuristik trivial yang konsisten, sehingga jaminan optimalitas solusi tetap berlaku.

## 4. Repositori GitHub

<https://github.com/ImmanuelTambunan/ecommerce-cs-ai-agent>



**Gambar 2.** Tampilan Repository pada pengerjaan Milestone 1

|                                                                                                                                                          |                                        |                  |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|------------------|----|
| Update README.md                                                                                                                                         | MutiaraSianturi authored 1 hour ago    | Verified 88a772e | <> |
| update readme                                                                                                                                            | MutiaraSianturi committed 1 hour ago   | 9550cd1          | <> |
| Merge branch 'main' of <a href="https://github.com/ImmanuelTambunan/ecommerce-cs-ai-agent">https://github.com/ImmanuelTambunan/ecommerce-cs-ai-agent</a> | MutiaraSianturi committed 2 hours ago  | 0bc8007          | <> |
| Create License                                                                                                                                           | MutiaraSianturi committed 2 hours ago  | 7530f11          | <> |
| update readme                                                                                                                                            | AdithyaSilaban committed 8 hours ago   | 11b3c0d          | <> |
| Add .gitignore                                                                                                                                           | MutiaraSianturi committed 8 hours ago  | 8117735          | <> |
| Merge branch 'main' of <a href="https://github.com/ImmanuelTambunan/ecommerce-cs-ai-agent">https://github.com/ImmanuelTambunan/ecommerce-cs-ai-agent</a> | MutiaraSianturi committed 13 hours ago | d7268b6          | <> |
| Add baseline UCS search and update README'                                                                                                               | MutiaraSianturi committed 13 hours ago | 108535b          | <> |
| Update README.md                                                                                                                                         | MutiaraSianturi authored 13 hours ago  | Verified 9cd5c77 | <> |
| Commits on Sep 15, 2026                                                                                                                                  |                                        |                  |    |
| Initial commit                                                                                                                                           | ImmanuelTambunan authored yesterday    | Verified 0f6832d | <> |

**Gambar 3.** Riwayat commit mencerminkan kontribusi bertahap dari seluruh anggota kelompok

Repositori telah dilengkapi dengan berkas README.md yang memuat deskripsi proyek dan panduan instalasi, konfigurasi lingkungan proyek menggunakan Astral uv melalui pyproject.toml, berkas .gitignore, LICENSE, serta struktur folder modular dengan kode program yang ditempatkan pada src/. Riwayat commit menunjukkan adanya kontribusi bertahap dalam pengembangan repositori oleh anggota kelompok.